МИНИСТЕРСТВО СЕЛЬСКОГО ХОЗЯЙСТВА РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ АГРАРНЫЙ УНИВЕРСИТЕТ - МСХА имени К. А. ТИМИРЯЗЕВА»

(ФГБОУ ВО РГАУ-МСХА имени К. А. Тимирязева)

Институт экономики и управления АПК
Кафедра статистики и кибернетики

Направление: 09.03.02 Информационные системы и технологии
Направленность: Большие данные и машинное обучение

КУРСОВОЙ ПРОЕКТ

на тему: «Приложение для составления и синхронизации списков дел (to-do list)».

Выполнил:
студент 2 курса группы ДЭ 21-25
Галеев Султан Азатович

Дата регистрации КП

на кафедре: _____

Допущен к защите

Руководитель:

_____
учёная степень, учёное звание, ФИО

Члены комиссии:

_____ _____
учёная степень, учёное звание, ФИО подпись

_____ _____
учёная степень, учёное звание, ФИО подпись

_____ _____
учёная степень, учёное звание, ФИО подпись

Оценка _____

Дата защиты _____

Москва 2026

РГАУ-МСХА имени К. А. Тимирязева
Институт экономики и управления АПК
Кафедра статистики и кибернетики

ЗАДАНИЕ НА КУРСОВОЙ ПРОЕКТ

Обучающийся: Галеев Султан Азатович

Группа: ДЭ 21-25

Дисциплина: «Разработка профессиональных приложений».

Тема: «Приложение для составления и синхронизации списков дел (to-do list)».

Исходные данные: два персональных компьютера, одна локальная сеть, язык Python, приложение с графическим интерфейсом, база SQLite.

Необходимо разработать: создание и редактирование задач, изменение отметки выполнения, поиск и фильтрацию записей, страницу статистики, сохранение списка в текстовый файл, сервер обмена данными и проверку работы двух клиентов.

Материалы к защите: исходные файлы проекта, пояснительная записка, схемы операций, снимки интерфейса, схемы работы приложения и результаты проверки запросов.

Дата выдачи задания: _____

Руководитель: _____

Задание принял: _____

АННОТАЦИЯ

В курсовом проекте рассмотрено приложение для ведения общего списка задач на двух персональных компьютерах. В графическом окне можно добавить задачу, исправить её название, изменить отметку о выполнении или удалить запись. Предусмотрены поиск по названию, фильтр по состоянию, страница со статистикой и сохранение списка в TXT. Оба компьютера используют один клиент PySide6, подключённый к серверу в локальной сети.

Сервер FastAPI принимает запросы и обращается к базе SQLite. Задачи хранятся в одной таблице, поэтому изменения доступны с обоих устройств. Клиент запрашивает список каждые две секунды и обновляет окно, когда данные изменились. Работа без соединения с сервером не предусмотрена.

Структура базы данных включает идентификатор, название и отметку выполнения задачи. Для обмена данными используются пять HTTP-маршрутов. Алгоритмы добавления, изменения и обновления задач представлены блок-схемами. Работа графического интерфейса и двух экземпляров клиента на одном компьютере показана на снимках экрана. Проверка двух физических компьютеров описана отдельно и не считается проведённой.

Ключевые слова: Python, PySide6, FastAPI, SQLite, список дел, синхронизация, настольное приложение.

Содержание

Введение **5**

Глава 1. Проектирование приложения для общего списка дел **6**

1.1 Предметная область и назначение программы 6

1.2 Пользовательские действия и требования 7

1.3 Выбор инструментов 10

1.4 Архитектура и обмен данными 12

1.5 Структура базы данных и интерфейса 14

Глава 2. Реализация и проверка приложения **17**

2.1 Состав проекта и запуск 17

2.2 Сервер и операции с данными 18

2.3 Алгоритмы работы программы 19

2.4 Пользовательский интерфейс 23

2.5 Проверка работы и сценарии приёмки 28

2.6 Ограничения программы 29

Заключение **31**

Список использованных источников **32**

Приложение А. Примеры входных и выходных данных **35**

Приложение Б. Порядок проверки на двух компьютерах **36**

Введение

При работе на домашнем и учебном компьютерах один и тот же список заданий приходится поддерживать на двух устройствах. Если записи хранятся отдельно, на одном ПК задача может быть уже выполнена, а на другом по-прежнему отображаться как незавершённая. Общая база позволяет видеть одни и те же задачи на обоих компьютерах.

В проекте разрабатывается настольное приложение для двух ПК. Каждый компьютер запускает графическое окно, в котором можно добавлять, исправлять, искать и отмечать задачи. Оба окна отправляют запросы одному серверу, который сохраняет задачи в базе SQLite. После изменения данных на одном компьютере второй получает новое состояние при следующем обновлении.

Программа рассчитана на два персональных компьютера в одной локальной сети. Для обоих устройств используется один настольный клиент. Сервер и файл базы данных располагаются на первом компьютере.

Цель курсового проекта: создать приложение для ведения общего списка задач с графическим интерфейсом, базой данных SQLite и синхронизацией между двумя ПК через локальную сеть.

Для достижения цели необходимо определить операции пользователя, создать таблицу базы данных, написать обработчики запросов, собрать интерфейс с тремя страницами и проверить обмен изменениями между двумя клиентами. Отдельно рассматриваются неправильный ввод и отсутствие сетевого соединения.

Объект проекта: общий список дел. **Предмет проекта**: хранение задач и передача изменений между двумя персональными компьютерами.

Графический интерфейс разработан на PySide6, HTTP-запросы обрабатывает FastAPI, а данные хранятся в SQLite. Взаимодействие этих компонентов позволяет изменять общий список задач из двух независимых окон.

Глава 1. Проектирование приложения для общего списка дел

1.1 Предметная область и назначение программы

Списки дел используются для записи учебных заданий, покупок, личных планов и других предстоящих дел. Для каждой задачи хранятся идентификатор, название и отметка выполнения. Например, задачу «Сдать практическую работу» можно добавить в общий перечень и отметить выполненной после завершения. Даты, приоритеты, категории и повторяющиеся события в рассматриваемую модель не входят. Для двух компьютеров достаточно общего перечня задач и отметок об их выполнении.

Пользователь может работать на двух компьютерах. При локальном хранении на каждом устройстве возникли бы два независимых списка. Даже если файлы имеют одинаковую структуру, они со временем расходятся: на одном компьютере задача отмечена выполненной, а на втором об этом ещё неизвестно. При ручном копировании файлов нужно отдельно следить, какая версия списка последняя. Список хранится в SQLite на первом компьютере; оба клиента обращаются к нему через FastAPI.

Приложение рассчитано на локальную сеть, например домашнюю или учебную. Один компьютер запускает сервер и хранит данные. Второй компьютер является клиентом. При этом на первом компьютере тоже можно запустить клиент, поэтому у обоих пользователей будет одинаковый интерфейс. Сведения о задачах передаются по HTTP в виде JSON-документов. Клиент отправляет запрос, сервер возвращает ответ; такой обмен описан в русскоязычном руководстве MDN [6]. JSON представляет данные в виде объектов, массивов и значений разных типов [18]. В ответе сервера передаются идентификатор, название и состояние задачи.

Основу серверного интерфейса составляют чтение списка, добавление записи, исправление её названия, переключение отметки и удаление. После каждой операции данные остаются в одной базе. Сервер отвечает за проверку запроса и изменение нужной строки, а клиенты - за отображе-

ние результата. Такой способ хранения позволяет работать с задачами по идентификатору, даже если две записи имеют одинаковое название.

Сохранение данных и синхронизация проверяются раздельно. Сохранение означает, что запись остаётся в SQLite после перезапуска сервера. Синхронизация означает, что изменение с одного компьютера получает второй клиент. Если задача видна на другом ПК, но исчезает после перезапуска сервера, обмен данными работает, а сохранение данных не работает. Поэтому отдельно проверяются чтение общей базы и повторный запуск с тем же файлом данных.

У программы есть несколько допущений. Компьютеры используют доверенную локальную сеть; в приложении нет личных аккаунтов. Пользователи видят один список, а не личные списки каждого. Клиенты подключаются к серверу по его локальному IP-адресу. Если компьютер-сервер выключен, клиент сохраняет на экране ранее полученный список, но не обновляет его. Нижняя строка сообщает об отсутствии связи; отправка новой команды завершится ошибкой.

1.2 Пользовательские действия и требования

Функции приложения определены по последовательности действий пользователя. При составлении требований учитываются действия, исходные данные и ожидаемый результат операции; такой способ описания используется при проектировании информационных систем [1, 2]. После открытия приложения пользователь видит список задач. Он может создать новую запись, исправить название, отобрать записи по тексту или состоянию, а также отметить выполнение. Для удаления требуется выбрать конкретную строку. Второй компьютер должен показывать тот же перечень, хотя обновление может произойти с небольшой задержкой. Эти действия сформировали набор функциональных требований, приведённый в таблице 1.1.

Таблица 1.1. Функциональные требования к приложению

Код	Требование	Проверяемый результат
F1	Показать актуальный перечень задач	Виден список записей
F2	Добавить задачу с непустым названием	Новая запись появилась
F3	Изменить отметку о выполнении	Состояние поменялось
F4	Удалить выбранную задачу	Запись исчезла
F5	Передать изменения второму клиенту	Список на ПК №2 обновился
F6	Сохранить задачи после перезапуска	Список остался в базе
F7	Обработать неверный ввод	Ошибочная запись не создана
F8	Показать проблему соединения	Пользователь видит сообщение
F9	Перейти между тремя страницами	Переключается раздел интерфейса
F10	Исправить название существующей задачи	Новое название сохранено
F11	Найти задачи и применить фильтр	Видны подходящие записи
F12	Посчитать выполненные задачи	Показаны числа и процент
F13	Сохранить список в TXT	Создан текстовый файл со всеми задачами

Ввод названия задачи ограничен 120 символами. То же правило действует при редактировании. Сервер проверяет текст независимо от кли-

ентского окна, поскольку запрос можно отправить и напрямую через страницу документации API. Название проверяется после удаления пробелов по краям строки; длина должна составлять от 1 до 120 символов. Управляющие символы (например, перевод строки и табуляция) сервер отклоняет: название должно отображаться одной строкой списка [19].

Признак выполнения принимает два значения. В интерфейсе выполненная задача отмечается знаком [x], невыполненная: знаком [ ]; при передаче по сети используется логическое значение. В списке не нужно выводить техническое значение поля `done`. Название записи остаётся одинаковым независимо от состояния. Повторное изменение переводит задачу из состояния «Выполнено» обратно в «В работе».

При редактировании название меняется только у выбранной записи; её идентификатор и отметка выполнения сохраняются. Поиск и фильтрация выполняются над уже полученным списком на клиенте и не изменяют содержимое базы данных. Кнопка «Сохранить в TXT» записывает все задачи в текстовый файл на компьютере пользователя; отдельный маршрут на сервере для этого не нужен. Для удаления пользователь должен выбрать строку списка. Если ни одна строка не выбрана, интерфейс показывает подсказку. На уровне сервера удаление выполняется по идентификатору. Когда такого идентификатора нет, сервер отвечает кодом 404, не изменяя другие записи. Это позволяет отделить отсутствие задачи от успешно проведённой операции. Семантика ошибок HTTP и назначение кодов ответа описаны в стандарте HTTP [17].

Нефункциональные требования касаются установки, работы с сетью и отображения ошибок. Программа должна запускаться без внешней учётной записи, а для хранения данных не должна требоваться установка отдельного серверного процесса базы данных. Все основные действия располагаются в одном окне. Для всех сетевых запросов установлен таймаут 2 секунды. Запросы выполняются в графическом потоке, поэтому при задержке сети окно может на короткое время перестать отвечать. Перечень нефункциональных требований приведён в таблице 1.2.

Таблица 1.2. Нефункциональные требования и ограничения

Код	Содержание
N1	Оба клиента имеют одинаковую функциональность и интерфейс
N2	Данные хранятся в одной базе на компьютере-сервере
N3	Рабочая сеть локальная; доступ в публичный интернет не требуется
N4	Интерфейс показывает проблемы ввода и соединения понятным текстом
N5	Проект устанавливается средствами Python и одного файла зависимостей
N6	Автоматическое обновление списка происходит раз в 2 секунды
N7	При демонстрации доступ к серверу ограничен доверенной локальной сетью

Для функциональных требований определены проверяемые сценарии. Для F2 достаточно создать новую запись и проверить её наличие. Для F5 нужны два клиента, подключённые к одной базе. Для F6 сервер перезапускается с тем же файлом SQLite. Для F7 отправляется пустое название, для F10 проверяется исправление названия с сохранением прежнего идентификатора. Работа поиска, фильтра и статистики проверяется в клиентском окне.

В базе нет поля с именем пользователя: любые изменения относятся к общему списку. Клиент поэтому не показывает форму входа и не разделяет задачи на личные и чужие.

1.3 Выбор инструментов

Приложение разделено на клиентскую и серверную части. На клиенте выполняется графический интерфейс. Сервер принимает запросы двух клиентов и работает с общей базой данных. Обе части написаны на Python. Для обработки запросов, работы с интерфейсом и обращения к базе применяются библиотеки этого языка. Для обработки строк, списков и функций используются обычные средства Python; их применение рассматривается в учебном пособии по Python 3 [3]. Доступ к SQLite предоставляется стандартным модулем `sqlite3` [7].

Для интерфейса используется PySide6, официальный набор привязок Qt для Python. С его помощью можно создавать окно, текстовое поле, кнопки, список и разные страницы без написания HTML или JavaScript.

Окно и его страницы построены из стандартных элементов Qt. В пособии Н. А. Прохоренка и В. А. Дронова рассматривается создание окон и обработка событий в Qt; в данной работе использованы привязки PySide6, для которых доступна отдельная документация [5, 12]. В окне задействованы поле ввода, список, кнопки, выпадающий фильтр, индикатор выполнения и три страницы.

Одинаковый исходный код клиента PySide6 запускается на компьютерах с Windows. Оба клиента обращаются к общему серверу по локальной сети. Каждый клиент запрашивает данные самостоятельно, поэтому изменения списка передаются через сервер без копирования файлов базы данных.

Серверная часть построена на FastAPI. Схема обмена запросами соответствует клиент-серверному подходу, рассматриваемому в учебной литературе по веб-программированию на Python [4]. В данном проекте клиентом выступает окно PySide6, а не браузер. В FastAPI обработчик связывается с HTTP-методом и адресом. Клиент может запросить список задач по одному адресу, а создать новую задачу по другому. FastAPI автоматически формирует описание API и интерактивную страницу документации. Встроенная страница документации позволяет просмотреть доступные операции и вручную выполнить запрос к серверу [9].

Для передачи данных применён JSON. Запрос создания задачи содержит название. Ответ на чтение списка содержит идентификатор, текст и отметку выполнения каждой задачи. Тело запроса и ответа можно просмотреть в интерактивной документации FastAPI. FastAPI позволяет описать структуру тела запроса через модели данных, после чего проверяет типы перед вызовом обработчика [10]. На стороне Python для такой проверки используется Pydantic. Проверка названия выполняется при добавлении и редактировании: пробелы по краям удаляются, пустая строка отклоняется.

Для хранения задач используется SQLite. Это файловая база данных, с которой Python работает через встроенный модуль `sqlite3`. Файл базы находится на компьютере с FastAPI. Отдельный сервер СУБД не нужен. В проекте одна таблица с названиями и состояниями задач. SQL-запросы выполняются с параметрами вместо подстановки введённого пользова-

телем текста. Такой способ не требует ручного экранирования ввода [7]. Назначение технологий указано в таблице 1.3.

Таблица 1.3. Назначение использованных технологий

Инструмент	Роль в системе
Python	Общий язык клиентской и серверной частей
PySide6	Окно с тремя страницами, поиск, фильтр и индикатор выполнения
FastAPI	Маршруты запросов, проверка входных данных, документация API
SQLite	Хранение задач в файле на компьютере-сервере
Requests	Отправка HTTP-запросов из настольного клиента
HTTP и JSON	Передача запросов и данных между клиентами и сервером

Для отправки запросов из настольного клиента используется Requests. В этой библиотеке есть методы для обычных HTTP-операций и возможность устанавливать время ожидания. При ошибках соединения или при ответе сервера с кодом ошибки вызываются исключения. По полученному исключению клиент различает успешный запрос и сбой связи [16]. Для проверки серверных запросов применялся TestClient из набора средств FastAPI [11].

1.4 Архитектура и обмен данными

На рисунке 1.1 показаны два компьютера. На первом работают сервер FastAPI и клиент PySide6, на втором - только клиент. Оба клиента обращаются к одному адресу сервера, который выполняет операции с файлом SQLite. Клиентам не требуется открывать базу данных через сетевую папку.

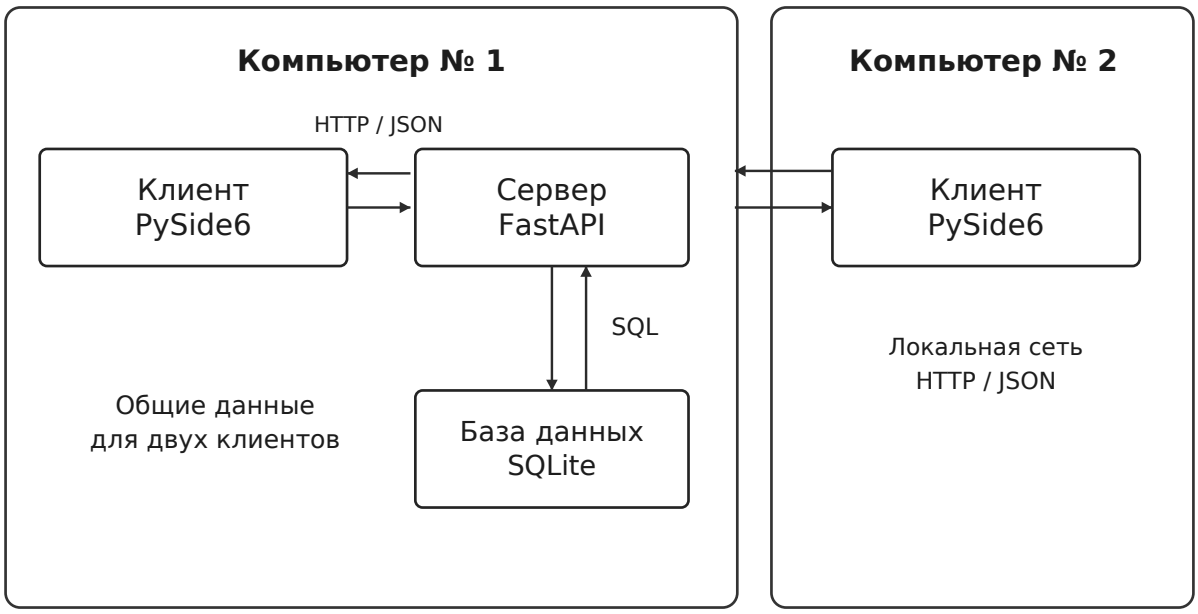


Рисунок 1.1. Взаимодействие двух клиентов, сервера и базы данных

Действие пользователя проходит несколько этапов. Например, при добавлении задачи клиент получает текст из поля ввода. Если введены только пробелы, сообщение об ошибке показывается сразу, без обращения к серверу. Если ввод корректный, клиент отправляет POST-запрос. Сервер ещё раз проверяет название, выполняет вставку в SQLite, сохраняет изменение и отправляет ответ об успехе. После ответа клиент запрашивает обновлённый список. Второй компьютер увидит эту запись после следующего запроса обновления.

При изменении состояния сначала выбирается конкретная строка. Из её данных извлекается целочисленный идентификатор, а затем отправляется PATCH-запрос по её идентификатору. Сервер переключает отметку выполнения соответствующей записи. Если запись уже отсутствует, сервер возвращает HTTP 404. Клиент показывает предупреждение «Не удалось выполнить действие» и повторно запрашивает список. Тот же принцип используется при удалении: операция выполняется только для существующего идентификатора. Смысл используемых методов и кодов ответа соответствует общим правилам HTTP [17].

Для синхронизации выбран периодический опрос сервера. В каждом клиенте таймер срабатывает один раз в две секунды и запускает получение списка задач. Полученный список сравнивается с предыдущим. Если изменений нет, интерфейс остаётся таким же. Если данные отличаются,

список заполняется заново. Применение таймера для периодических действий предусмотрено библиотекой Qt [15]. Постоянное соединение с сервером для этого алгоритма не используется.

Изменение появляется на втором ПК при следующем запросе таймера, поэтому обновление не происходит мгновенно. Если компьютер-сервер недоступен, клиент показывает ошибку и не сохраняет отправленную команду локально. После восстановления соединения очередной запрос возвращает актуальный список SQLite. Измерение оптимального интервала опроса не проводилось.

При одновременной работе двух пользователей возможны ситуации, когда один удаляет задачу, а второй пытается изменить её состояние. Проверка количества изменённых строк на сервере позволяет отличить несуществующую запись от существующей. В таком случае клиент получает 404. Правила сложного разрешения конфликтов не требуются: все изменения проходят через одну таблицу и выполняются как отдельные операции. Но приложение не фиксирует историю действий, поэтому после удаления нельзя восстановить прежнее состояние средствами самой программы.

1.5 Структура базы данных и интерфейса

Задачи хранятся в таблице `tasks`. В неё входят идентификатор, название, отметка выполнения. Идентификатор формируется автоматически и позволяет выбрать нужную запись при изменении или удалении. Название хранится как текст. Признак выполнения записывается как число 0 или 1, но в передаваемом JSON преобразуется в логическое значение. Отдельного поля владельца задачи нет, поскольку список общий.

SQLite позволяет создавать таблицу с первичным ключом и ограничениями на отсутствие значения NULL. В таблице нет внешних ключей: каждая запись содержит все данные, необходимые клиенту. Операция создания таблицы выполняется при запуске сервера, причём используется вариант с проверкой существования таблицы. Если файл базы уже содержит данные, повторный запуск не должен удалять их. Синтаксис создания таблицы и назначение первичного ключа описаны в документации SQLite [8]. Состав полей приведён в таблице 1.4.

Таблица 1.4. Поля таблицы tasks и их смысл

Поле	Тип	Назначение
id	INTEGER	Уникальный идентификатор записи
title	TEXT	Краткое название задачи
done	INTEGER	0: в работе, 1: выполнено

Для чтения списка сервер использует запрос SELECT с сортировкой. Новые записи выводятся выше старых, независимо от отметки выполнения. Это правило применяется сервером, поэтому оба клиента получают одинаковый порядок. SQL-операция SELECT предназначена для выбора данных из таблиц и допускает указание порядка результатов [7].

Изменение состояния выполняется операцией UPDATE, которая меняет поле done только у строки с указанным идентификатором [7]. Удаление реализуется операцией DELETE с таким же условием выбора одной записи [7]. Пользовательский текст передаётся в SQL как параметр, что позволяет безопасно хранить обычные символы, включая кавычки. В коде нет ручного построения SQL-запроса из введённого названия.

В клиенте три страницы (рисунок 1.2). На первой доступны добавление, поиск, фильтрация, редактирование, изменение статуса, удаление и сохранение списка в TXT. На второй показаны число всех задач, количество выполненных и индикатор доли выполнения. Третья содержит сведения о программе и адрес сервера. Переключение выполняется вкладками в верхней части окна.



Рисунок 1.2. Переход между страницами приложения

Для переключения трёх страниц используется QTabWidget [13]. Список построен на QListWidget: у каждой строки отдельно сохраняется

идентификатор задачи. Кнопка «Изменить» открывает стандартный диалог ввода текста; двойной щелчок по строке меняет отметку выполнения [14]. Поиск сравнивает названия без учёта регистра, а фильтр оставляет все задачи, только невыполненные или только выполненные.

В памяти клиента хранится последний полученный список задач. Это позволяет не перестраивать список каждые две секунды без необходимости. При обновлении, которое действительно меняет строки, клиент пытается сохранить выбранную пользователем запись по её идентификатору. Такой механизм нужен, чтобы автоматический опрос не мешал нажимать кнопки. Данные о выбранной строке читаются из специального поля элемента списка, а не извлекаются из видимого текста, который может совпадать у разных задач.

Страница статистики показывает общее число задач, количество выполненных и их процент. Если список пуст, индикатор показывает ноль. Эти показатели вычисляются из полученных клиентом данных, без дополнительных таблиц и запросов к базе. Адрес сервера задаётся при запуске клиента, а не через отдельную страницу настроек.

Глава 2. Реализация и проверка приложения

2.1 Состав проекта и запуск

Серверная часть содержит пять маршрутов HTTP и операции с SQLite. Настольный клиент выводит окно PySide6 и обрабатывает действия пользователя. Для установки библиотек предусмотрен файл зависимостей, для запуска на Windows используются командные файлы `START.bat`, `SERVER_ONLY.bat` и `CLIENT_OTHER_PC.bat`. Состав основных файлов приложения приведён в таблице 2.1.

Таблица 2.1. Основные файлы приложения

Файл	Назначение
server.py	Сервер FastAPI и хранение задач в SQLite
client.py	Окно PySide6, список дел и обработка кнопок
requirements.txt	Библиотеки для установки на двух компьютерах
START.bat	Запуск сервера и двух клиентов на одном ПК
SERVER_ONLY.bat	Запуск сервера на первом ПК
CLIENT_OTHER_PC.bat	Запуск клиента на втором ПК
README.txt	Инструкция по запуску
tasks.db	Файл SQLite с общим списком задач; создаётся при первом запуске

На обоих компьютерах устанавливается Python и библиотеки из файла `requirements.txt`. На первом ПК командой `START.bat` запускаются сервер и два клиента для локальной проверки. Для работы в сети на первом ПК открывают `SERVER_ONLY.bat`, а на втором `CLIENT_OTHER_PC.bat` и вводят IP-адрес первого компьютера. Сам сервер использует порт 8000. Клиент первого ПК может обращаться к адресу `127.0.0.1`; второму нужен реальный локальный IPv4-адрес первого ПК.

Чтобы узнать адрес, на первом компьютере достаточно выполнить команду `ipconfig`. Во время работы оба компьютера должны находиться в одной локальной сети. Если сервер не открывается со второго ПК, нужно проверить адрес и правила брандмауэра Windows. Важно не запускать от-

дельный сервер на каждом компьютере: иначе программы окажутся подключены к разным базам, и синхронизации не будет.

После запуска сервера рядом с `server.py` создаётся файл `tasks.db`, если его ещё нет. В нём сохраняются названия задач и отметки их выполнения. После остановки сервера файл остаётся на диске. Когда сервер запускается снова, он открывает ту же базу. SQLite работает с файлом базы непосредственно, без отдельного сервера СУБД.

2.2 Сервер и операции с данными

Сервер FastAPI обрабатывает пять маршрутов. При чтении он выбирает записи из таблицы `tasks`; при добавлении вставляет строку; отдельный PATCH-запрос исправляет название, другой PATCH меняет отметку выполнения; DELETE удаляет запись. Перечень операций представлен в таблице 2.2.

Таблица 2.2. HTTP-операции сервера

Метод	Адрес	Назначение
GET	/tasks	Получить общий список задач
POST	/tasks	Добавить задачу
PATCH	/tasks/{id}	Поменять отметку выполнения
PATCH	/tasks/{id}/title	Исправить название задачи
DELETE	/tasks/{id}	Удалить выбранную задачу

Для новой задачи клиент передаёт JSON-объект с полем `title`. FastAPI преобразует данные к модели `Task`, после чего запускается обработчик добавления [10]. Он удаляет пробелы по краям и проверяет длину названия. Строка из одних пробелов и название длиннее 120 символов не записываются. Если ввод неверен, сервер возвращает HTTP 400 и сообщение об ошибке. Такая проверка нужна не только графическому окну: запрос может быть отправлен напрямую через страницу `/docs`.

Данные размещаются в одной таблице SQLite с полями `id`, `title`, `done`. Первичный ключ `id` генерируется автоматически. При добавлении выполняется SQL INSERT с параметром вместо подстановки пользовательского текста в запрос. Это позволяет хранить строки, содержащие кавычки и другие символы, без ручного формирования SQL-выражения

[7]. После сохранения сервер возвращает ответ, а клиент запрашивает обновлённый перечень.

Чтобы исправить название, клиент открывает диалог ввода и отправляет PATCH-запрос на адрес `/tasks/{id}/title`. На сервере применяется та же проверка текста, что при создании задачи. Меняется только поле `title`; первичный ключ и состояние остаются прежними. Когда пользователь выбирает задачу и нажимает кнопку изменения состояния, приложение отправляет другой PATCH-запрос с идентификатором. Сервер выполняет SQL UPDATE, переключающий число в поле `done` между 0 и 1. Чтобы изменить именно выбранную запись, используется условие `WHERE id = ?` [7]. При удалении аналогичным образом выполняется SQL DELETE [7]. Если идентификатор отсутствует, сервер возвращает ошибку HTTP 404.

При чтении списка выполняется запрос SELECT с сортировкой по идентификатору: новые задачи располагаются первыми. В ответе каждая запись содержит целочисленный идентификатор, название и логическую отметку выполнения. Клиент использует эти значения при построении списка. Ему не требуется знать расположение файла базы данных или выполнять SQL-запросы самостоятельно.

Блок `with sqlite3.connect` автоматически завершает транзакцию: сохраняет изменения при успешной операции и откатывает их при ошибке.

2.3 Алгоритмы работы программы

Алгоритм добавления задачи начинается с пользовательского ввода. После нажатия кнопки выполняется проверка названия. Пустую строку нельзя передать серверу как корректную задачу. Для остальных строк отправляется HTTP-запрос. Если соединение недоступно или сервер отвечает ошибкой, интерфейс сообщает об этом пользователю, не очищая введённый текст. При успешном ответе поле ввода очищается, затем выполняется чтение актуального списка. Последовательность приведена на рисунке 2.1.

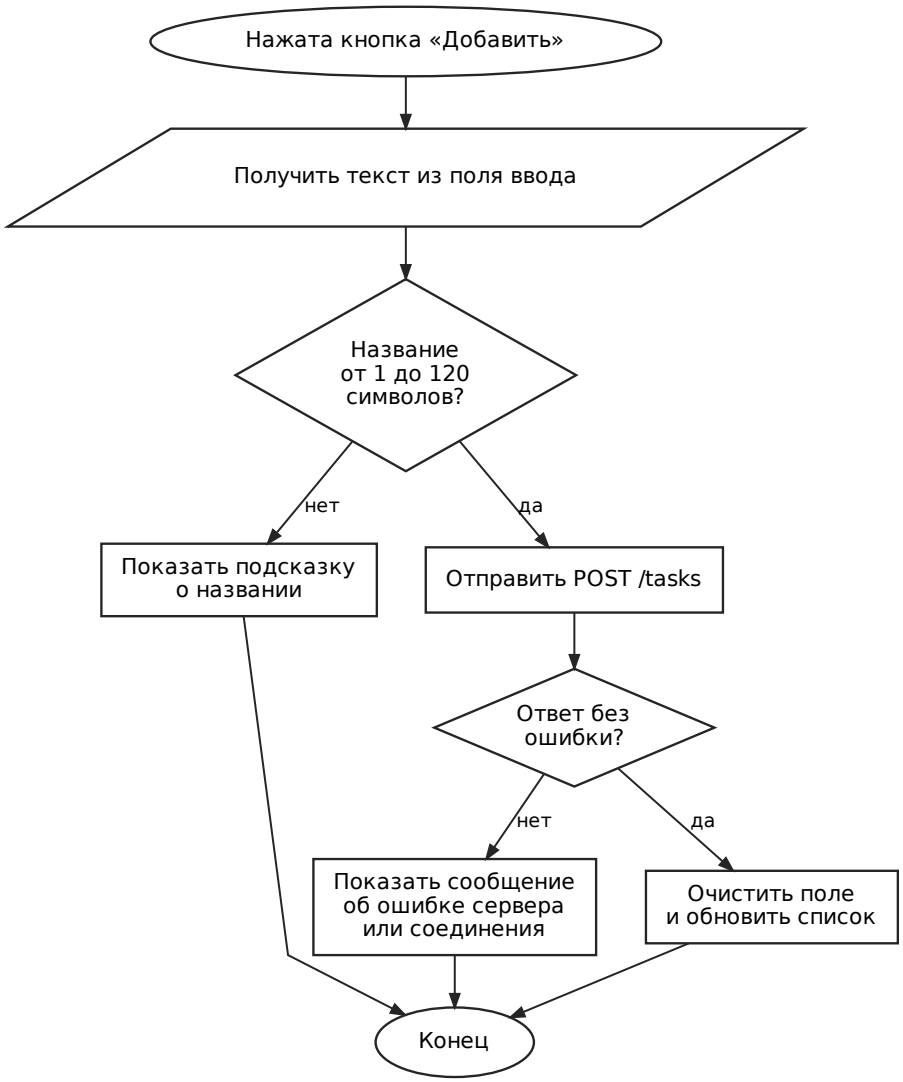


Рисунок 2.1. Последовательность добавления задачи и обработки ошибки

На блок-схемах эллипс обозначает начало и конец алгоритма, прямоугольник обозначает действие, а ромб обозначает проверку условия [20]. Ветви ошибки заканчиваются сообщением пользователю без добавления записи.

Редактирование названия показано на рисунке 2.2. Если задача не выбрана, пользователь получает подсказку. Нажатие «Отмена» закрывает диалог без изменений. Название проверяется перед отправкой PATCH-запроса; после успешного ответа клиент получает обновлённый список.

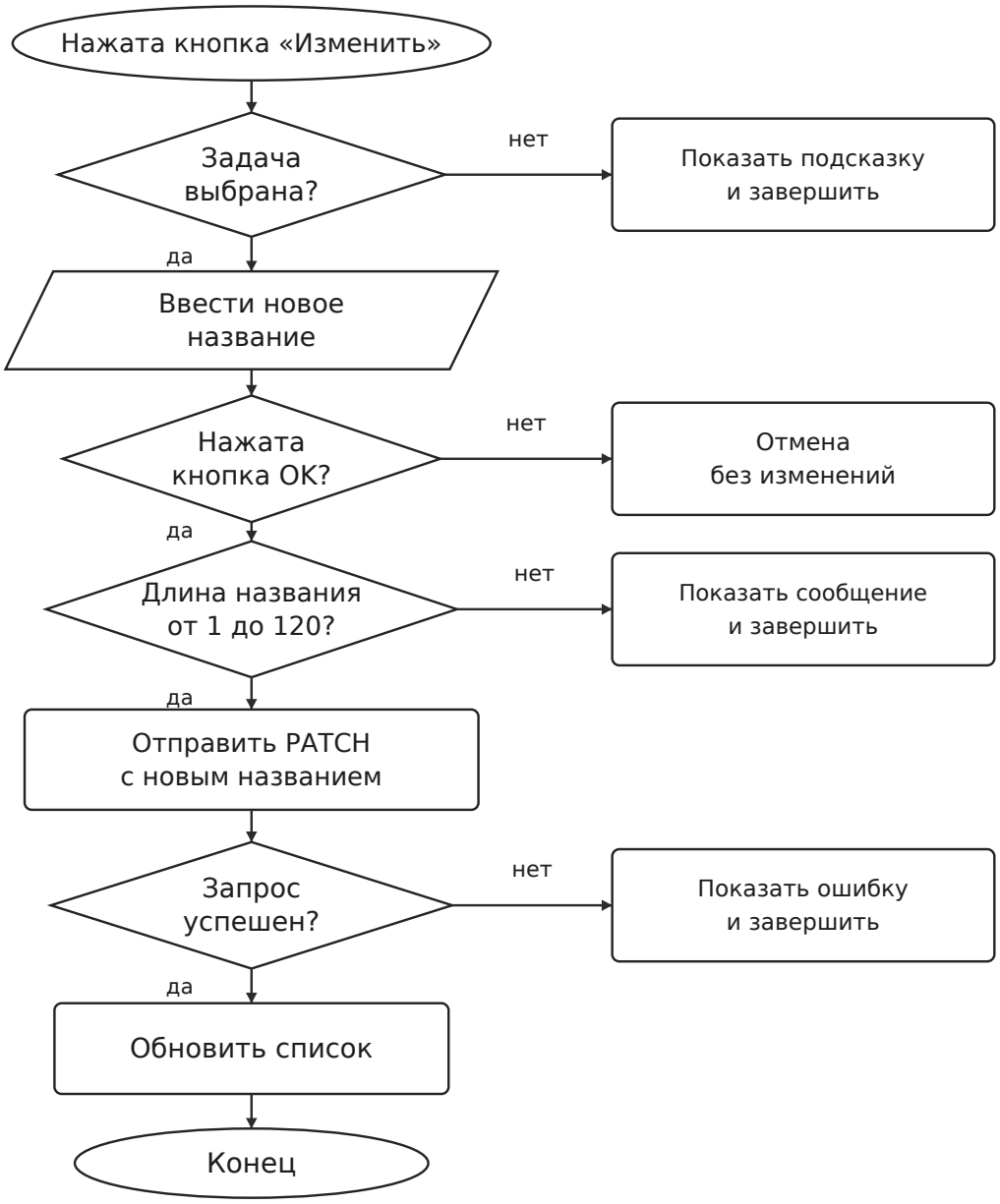


Рисунок 2.2. Алгоритм изменения названия задачи

Работа таймера показана на рисунке 2.3. Раз в две секунды клиент запрашивает список. Если данные изменились, он обновляет перечень и показатели статистики; иначе оставляет экран без изменений. Если чтение не удалось из-за отсутствия соединения, нижняя строка сообщает о потере связи. Ранее полученные записи остаются видимыми. При следующем срабатывании таймера запрос повторяется.

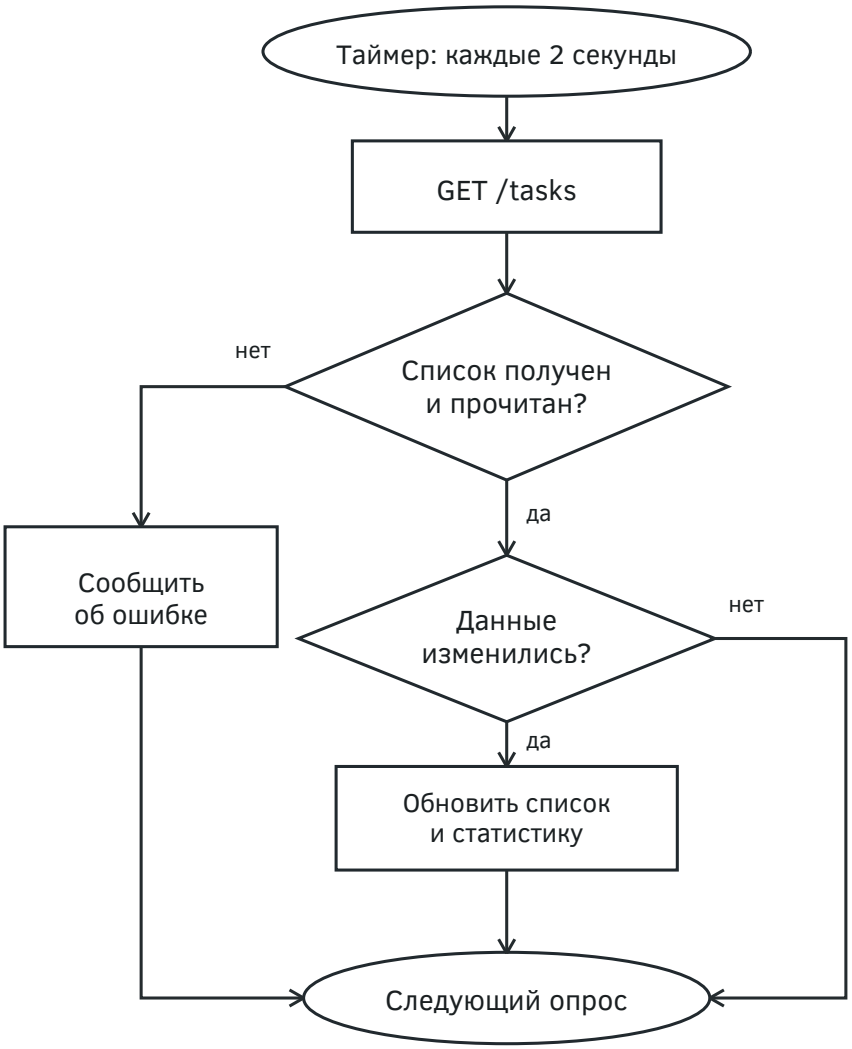


Рисунок 2.3. Алгоритм периодического обновления списка задач

Выбранная схема не требует отдельного канала уведомлений от сервера. Оба клиента сами обращаются за актуальными данными. У такого способа есть ограничение: фактическое появление изменения зависит от момента следующего опроса. Если задача была создана непосредственно после очередного запроса второго клиента, он увидит её только при следующем обращении. Таймер настроен на две секунды. Производительность и оптимальная частота запросов отдельно не измерялись.

При запуске окно отправляет запрос списка. При недоступном сервере появляется сообщение «Нет связи с сервером». При первом запуске экран остаётся пустым, а после временного отключения виден ранее полученный список. Если сервер вернул неверный JSON, окно также показывает сообщение об отсутствии связи. Очередной успешный запрос заменяет прежние данные.

Навигация между тремя страницами не затрагивает базу данных. Вкладки «Задачи», «Статистика» и «О программе» выбирают соответствующий экран. Для этого используется QTabWidget, который показывает нужную страницу в том же окне [13]. Состояние соединения и таймер принадлежат главному окну, поэтому переход на информационную страницу не останавливает периодическое обновление. При возвращении пользователь видит последнее полученное состояние задач.

Обработка кнопок строится на сигнале нажатия. Qt связывает действия пользователя с вызываемыми методами; для приложения на Python это обычные функции объекта окна. В отличие от самостоятельного цикла опроса клавиатуры, библиотека управляет событиями и вызывает нужный обработчик при нажатии. Обработка сигналов и событий относится к стандартной модели Qt [5, 12].

Алгоритм удаления проверяет выбранную строку прежде, чем отправить запрос. Идентификатор хранится внутри элемента списка как отдельные данные. Такое решение защищает от неоднозначности: два пункта могут иметь одинаковые названия, но разные идентификаторы. Операция выполняется именно над выбранной записью. Текст названия не используется для удаления, поэтому повторяющиеся формулировки задач не приводят к случайному изменению нескольких строк.

2.4 Пользовательский интерфейс

Главное окно содержит три вкладки: «Задачи», «Статистика» и «О программе». На вкладке «Задачи» расположены поле для названия, кнопка добавления, поиск, фильтр и список записей. Выбранную задачу можно переименовать, отметить выполненной, вернуть в работу или удалить. Кнопка «Сохранить TXT» записывает весь список в текстовый файл. На рисунке 2.4 показано главное окно программы в Windows.

Для поиска достаточно ввести часть названия. Фильтр «Все», «В работе» или «Выполнено» ограничивает видимые записи, но не изменяет данные на сервере. Перед названием показано [x] для выполненной задачи и [ ] для невыполненной. На снимке показаны пять задач, две из них отмечены выполненными. Среди записей есть «Запись к врачу», «Домашняя работа» и «Уборка по дому».

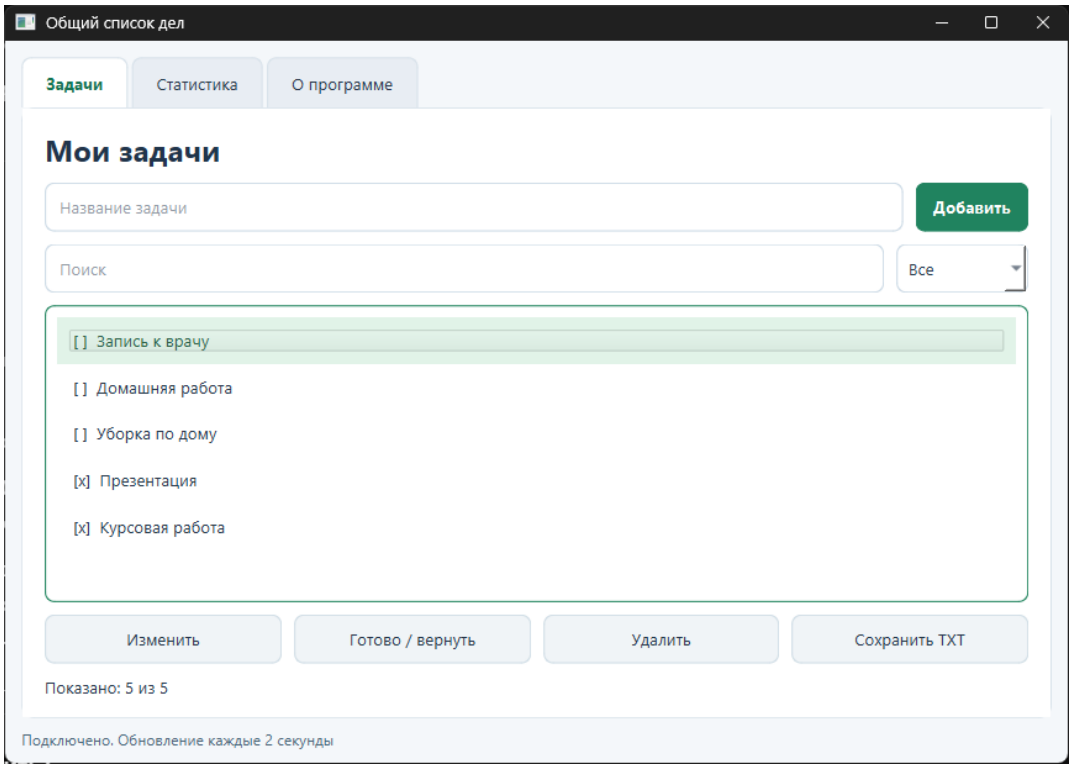


Рисунок 2.4. Окно приложения со списком задач

Редактирование выполняется через стандартный диалог ввода текста QInputDialog. Он открывается после выбора строки и нажатия кнопки «Изменить». На рисунке 2.5 показано изменение названия первой задачи на «Запись к врачу в 19:00».

При пустом названии окно показывает подсказку. После подтверждения редактирования клиент отправляет запрос на сервер и перечитывает список. Если задача не выбрана, изменение не выполняется. Удаление также требует выбора записи и подтверждения действия.

На вкладке «Статистика» отображаются три числа: количество всех задач, задач в работе и выполненных. Шкала показывает долю выполненных. Значения рассчитываются из полученного списка; отдельного запроса к серверу для статистики нет. На рисунке 2.6 показаны три блока статистики и шкала прогресса. В списке пять задач: три в работе и две выполнены, доля выполнения составляет 40%.

На вкладке «О программе» показаны назначение приложения, способ хранения данных, период обновления и адрес сервера (рисунок 2.7). Переключение между страницами выполнено с помощью QTabWidget [13].

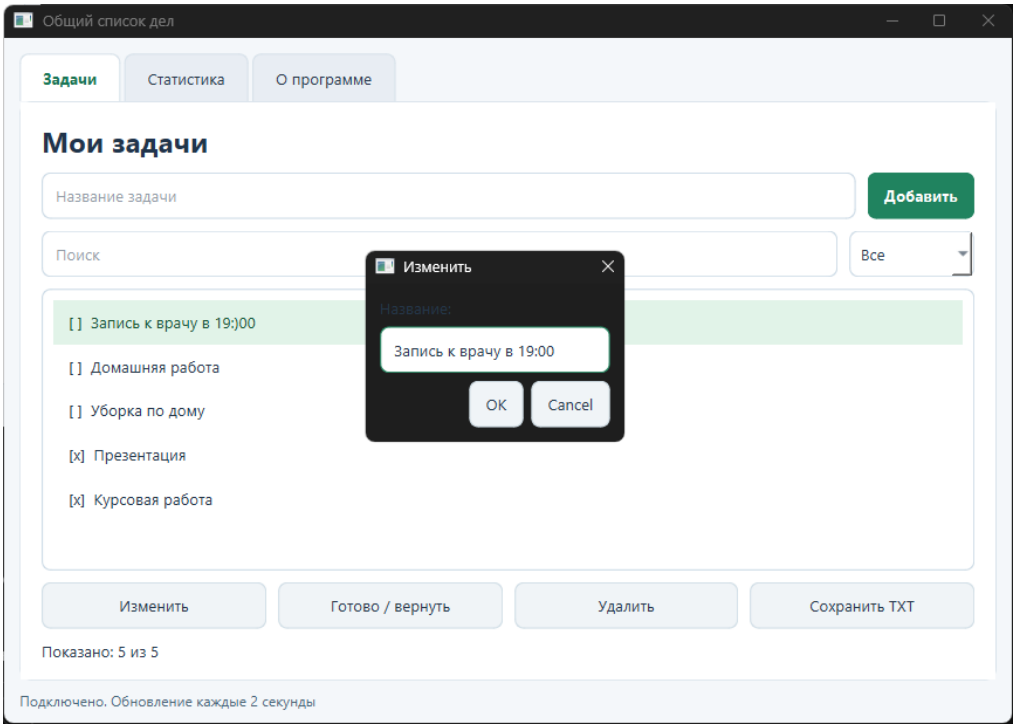


Рисунок 2.5. Диалог изменения названия выбранной задачи

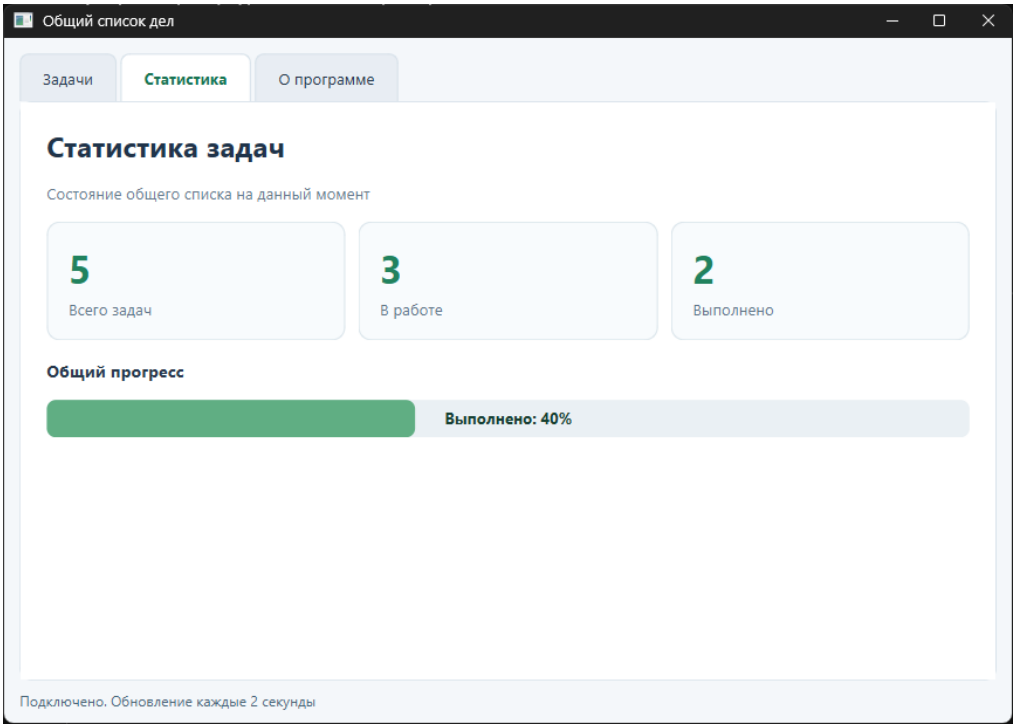


Рисунок 2.6. Вкладка «Статистика» в работающем приложении

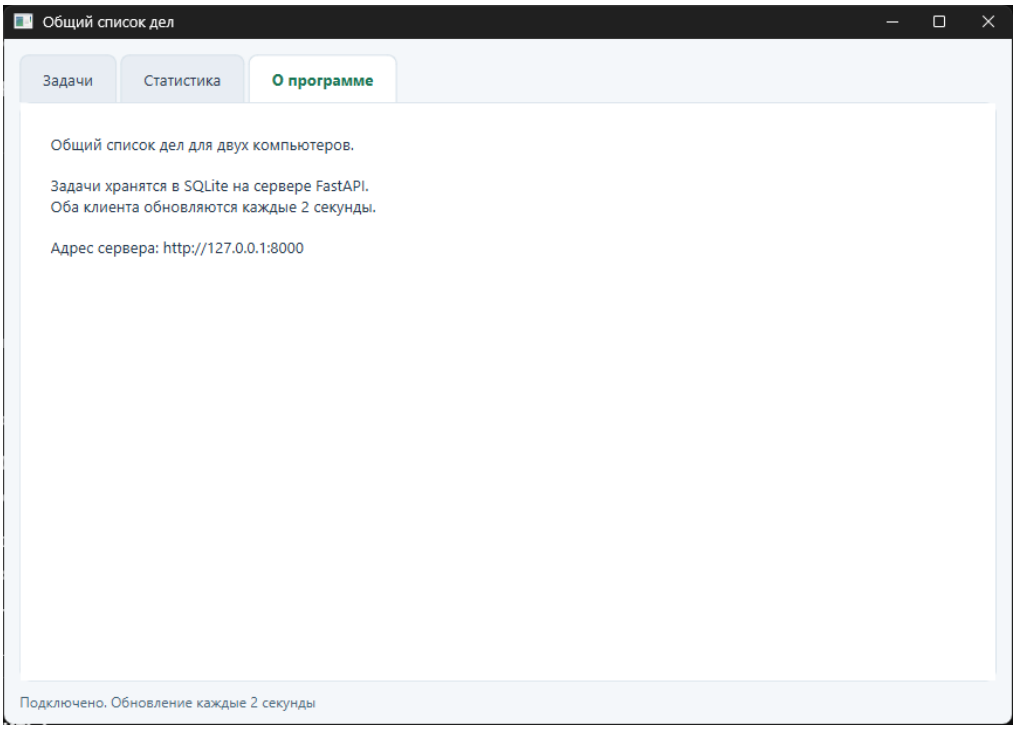


Рисунок 2.7. Вкладка «О программе» в работающем клиенте

Начальный размер окна составляет 780 на 530 пикселей. Элементы размещены в вертикальных и горизонтальных компоновщиках; при изменении размера окна область списка растягивается. Для вывода используется QListWidget: он позволяет связать строку с идентификатором задачи [14]. Благодаря этому одинаковые названия не мешают выбирать конкретную запись. Выделение сохраняется при обновлении списка, если выбранная задача ещё существует.

Для проверки совместной работы на одном компьютере запущены два отдельных окна клиента. После переименования задачи оба окна показывают запись «Запись к врачу в 19:00» (рисунок 2.8). Этот снимок подтверждает запуск двух экземпляров приложения и общее состояние списка на одном ПК; испытание связи между двумя физическими компьютерами проводится отдельно.

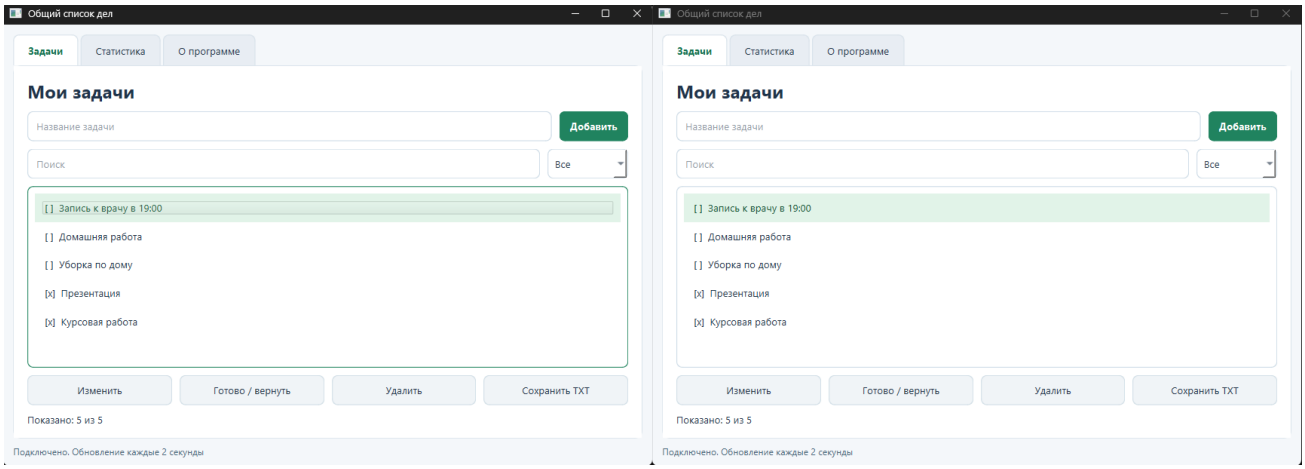


Рисунок 2.8. Два клиента, запущенные на одном компьютере

Экспорт в TXT выполняется на компьютере пользователя. Программа записывает каждую задачу с отметкой [x] либо [ ] в отдельную строку. Используется кодировка UTF-8 с меткой BOM для корректного чтения русского текста в Блокноте Windows. На рисунке 2.9 показан файл, сохранённый из приложения и открытый в текстовом редакторе. Экспорт не меняет данные в SQLite.

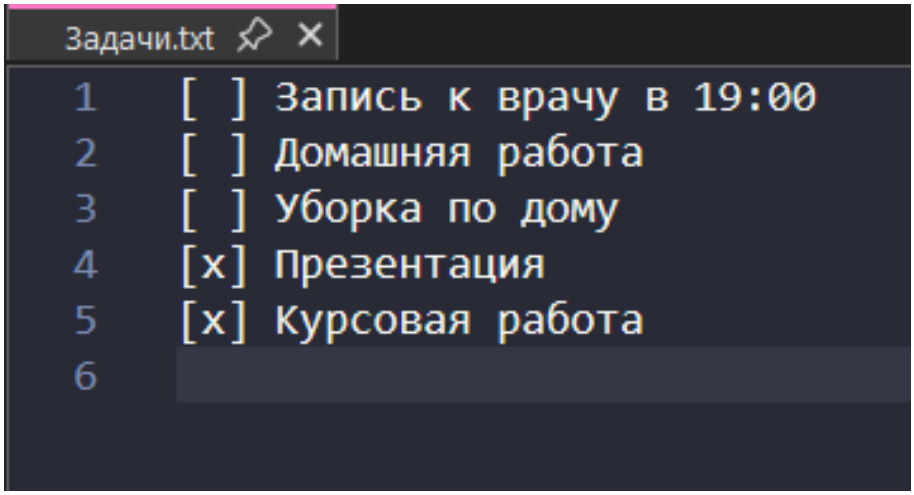


Рисунок 2.9. Содержимое файла TXT после экспорта списка задач

Если пользователь отменяет выбор файла, запись не выполняется. При ошибке сохранения появляется сообщение. При потере соединения с сервером клиент оставляет на экране последний полученный список и показывает в нижней строке состояние подключения. После восстановления связи список обновляется при следующем запросе.

2.5 Проверка работы и сценарии приёмки

Сервер проверялся на временной базе, не связанной с пользовательскими данными. Два экземпляра TestClient обращались к одной таблице и проверяли добавление, редактирование, переключение состояния, удаление и неверный ввод [11]. Затем тот же набор основных операций выполнялся через реально запущенный HTTP-сервер с двумя независимыми сеансами Requests. Проверки охватывали успешные операции, ошибочный ввод и сохранение записей. Эти испытания не использовали виджеты PySide6: проверялся обмен по HTTP и содержимое SQLite. Основные сценарии и ответы сервера приведены в таблице 2.3.

Таблица 2.3. Проверка основных HTTP-операций

№	Проверяемое действие	Результат
1	Получение пустого списка до добавления	HTTP 200; пустой список
2	Создание задачи с русским названием	HTTP 200
3	Чтение новой задачи вторым тестовым клиентом	Запись найдена
4	Редактирование названия и чтение первым клиентом	Новое название получено
5	Изменение состояния и чтение первым клиентом	`done = true`
6	Отклонение названия из пробелов	HTTP 400
7	Ошибка для отсутствующего идентификатора	HTTP 404
8	Удаление и повторное чтение	Запись отсутствует
9	Открытие документации API	HTTP 200
10	Перезапуск HTTP-сервера с тем же файлом базы	Данные сохранены

В таблице приведены операции серверной части. Поиск, фильтр, статистика и экспорт в TXT выполняются в клиентском окне и проверяются отдельно на Windows-ПК.

Один из сценариев включал следующую последовательность действий. Сначала сервер возвращает пустой список. Первый тестовый

клиент отправляет название «Подготовить курсовую», после чего второй читает список и получает эту задачу. Второй клиент исправляет название и меняет состояние задачи на выполненное, а первый получает оба изменения при следующем чтении. После удаления оба видят пустой перечень. В этом примере проверяется совместное использование базы, а не скорость передачи данных.

Помимо основных операций проверена обработка неправильных запросов. В таблице 2.4 приведены результаты повторной проверки, выполненной на временной базе без пользовательских данных.

Таблица 2.4. Результаты проверки входных данных и запросов

Проверяемое условие	Запрос	Фактический ответ
Название из пробелов	POST /tasks	HTTP 400
Название из 121 символа	POST /tasks	HTTP 400
Перевод строки и табуляция	POST /tasks	HTTP 400
Несуществующий идентификатор	PATCH /tasks/999999	HTTP 404
Пустое название при редактировании	PATCH /tasks/1/title	HTTP 400
Удаление отсутствующей задачи	DELETE /tasks/999999	HTTP 404
Название с апострофом	POST /tasks	HTTP 200, запись создана
Повторение названия	POST /tasks	HTTP 200, отдельная запись

Проверки серверной части включали правильный ввод, ошибочные запросы, редактирование, отметки выполнения, удаление, получение изменений вторым HTTP-клиентом и сохранение SQLite после остановки сервера. В ходе проверки в Windows получены снимки главного экрана, статистики, редактирования, информационной вкладки, двух открытых клиентов и экспортированного текстового файла. Два окна работали на одном компьютере; проверка сетевого соединения между двумя физическими ПК остаётся отдельным сценарием из приложения Б.

2.6 Ограничения программы

Приложение не рассчитано на работу в публичной сети. На сервере отсутствуют вход пользователей и распределение прав. HTTP-запросы не

шифруются, поэтому сервер следует запускать в доверенной локальной сети, не открывая порт для подключения из интернета.

При потере связи клиент оставляет на экране последний полученный список и сообщает, что связь с сервером отсутствует. Неотправленные действия не сохраняются. Текст новой задачи остаётся в поле ввода, чтобы пользователь мог повторить отправку после восстановления соединения. Следующий успешный опрос перечитает список.

Заключение

Подготовлено приложение, в котором два настольных клиента обращаются к общему списку задач. Данные находятся в базе SQLite на компьютере-сервере; пользователям не приходится копировать файлы между устройствами.

Реализованы добавление, редактирование и удаление задач, переключение отметки выполнения, поиск и фильтрация полученного списка, а также его сохранение в TXT. В окне PySide6 три страницы: задачи, статистика и сведения о программе. Клиент проверяет обновления через FastAPI с периодом две секунды.

Проверены редактирование названий, передача изменений между независимыми HTTP-клиентами, неверный ввод и сохранение записей после перезапуска сервера. В Windows открыты два окна клиента с общим списком, проверены вкладка статистики, редактирование и экспорт TXT. Испытание взаимодействия между двумя физическими ПК не проводилось; порядок такой проверки приведён в приложении Б.

Для работы обоих клиентов требуется доступный компьютер-сервер. При отключении сети новые действия не сохраняются и должны быть повторены после восстановления соединения. При дальнейшем развитии программы сетевые запросы можно вынести из графического потока, чтобы окно не задерживалось при ожидании ответа.

Список использованных источников

1. Зараменских Е. П. Проектирование информационных систем : учебник и практикум для вузов. - 2-е изд. - Москва : Юрайт, 2025. - 119 с. - ISBN 978-5-534-21418-5. - URL: https://urait.ru/bcode/571331 (дата обращения: 04.11.2026).
2. Чистов Д. В., Мельников П. П., Золотарюк А. В., Ничепорук Н. Б. Проектирование информационных систем : учебник и практикум для вузов. - 2-е изд., перераб. и доп. - Москва : Юрайт, 2025. - 273 с. - ISBN 978-5-534-20361-5. - URL: https://urait.ru/bcode/560485 (дата обращения: 04.11.2026).
3. Полупанов Д. В., Абдюшева С. Р., Ефимов А. М. Программирование в Python 3 : учебное пособие. - Уфа : БашГУ, 2020. - 164 с. - ISBN 978-5-7477-5230-6. - URL: https://e.lanbook.com/book/179915 (дата обращения: 04.11.2026).
4. Янцев В. В. Web-программирование на Python : учебное пособие для вузов. - Санкт-Петербург : Лань, 2022. - 180 с. - ISBN 978-5-8114-9461-3. - URL: https://e.lanbook.com/book/233264 (дата обращения: 04.11.2026).
5. Прохоренок Н. А., Дронов В. А. Python 3 и PyQt 6. Разработка приложений. - Санкт-Петербург : БХВ-Петербург, 2023. - 832 с. - ISBN 978-5-9775-1706-5. - URL: https://library.cbr.ru/catalog/lib/books/404707/ (дата обращения: 04.11.2026).
6. MDN Web Docs. Обзор протокола HTTP [Электронный ресурс]. - URL: https://developer.mozilla.org/ru/docs/Web/HTTP/Guides/Overview (дата обращения: 04.11.2026).
7. Python Software Foundation. sqlite3 - DB-API 2.0 interface for SQLite databases [Электронный ресурс]. - URL: https://docs.python.org/3/library/sqlite3.html (дата обращения: 04.11.2026).

8. SQLite. CREATE TABLE [Электронный ресурс]. - URL: https://www.sqlite.org/lang_createtable.html (дата обращения: 04.11.2026).

9. FastAPI. First Steps [Электронный ресурс]. - URL: https://fastapi.tiangolo.com/tutorial/first-steps/ (дата обращения: 04.11.2026).

10. FastAPI. Request Body [Электронный ресурс]. - URL: https://fastapi.tiangolo.com/tutorial/body/ (дата обращения: 04.11.2026).

11. FastAPI. Testing [Электронный ресурс]. - URL: https://fastapi.tiangolo.com/tutorial/testing/ (дата обращения: 04.11.2026).

12. Qt Company. Qt for Python - Documentation [Электронный ресурс]. - URL: https://doc.qt.io/qtforpython-6/ (дата обращения: 04.11.2026).

13. Qt Company. PySide6.QtWidgets.QTabWidget [Электронный ресурс]. - URL: https://doc.qt.io/qtforpython-6/PySide6/QtWidgets/QTabWidget.html (дата обращения: 04.11.2026).

14. Qt Company. PySide6.QtWidgets.QListWidget [Электронный ресурс]. - URL: https://doc.qt.io/qtforpython-6/PySide6/QtWidgets/QListWidget.html (дата обращения: 04.11.2026).

15. Qt Company. PySide6.QtCore.QTimer [Электронный ресурс]. - URL: https://doc.qt.io/qtforpython-6/PySide6/QtCore/QTimer.html (дата обращения: 04.11.2026).

16. Requests. Quickstart [Электронный ресурс]. - URL: https://requests.readthedocs.io/en/latest/user/quickstart/ (дата обращения: 04.11.2026).

17. Fielding R., Nottingham M., Reschke J. HTTP Semantics. RFC 9110. - IETF, 2022. - URL: https://datatracker.ietf.org/doc/html/rfc9110 (дата обращения: 04.11.2026).

18. Bray T. The JavaScript Object Notation (JSON) Data Interchange Format. RFC 8259. - IETF, 2017. - URL: https://datatracker.ietf.org/doc/html/rfc8259 (дата обращения: 04.11.2026).

19. OWASP Cheat Sheet Series. Input Validation Cheat Sheet [Электронный ресурс]. - URL: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html (дата обращения: 04.11.2026).

20. ГОСТ 19.701-90 (ИСО 5807-85). Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения. - Росстандарт. - URL: https://protect.gost.ru/gost/details/0c85e066-f89e-433d-a6d3-b7f746bd0c7c (дата обращения: 04.11.2026).

Приложение А

Примеры входных и выходных данных

В таблице А.1 приведены примеры действий в интерфейсе. Входными данными служат название задачи и выбор записи или команды. Результатом становится изменение списка, статистики или сообщение пользователю.

Таблица А.1. Примеры действий пользователя и результатов

Действие	Входные данные	Ожидаемый результат
Добавление	«Сдать практическую работу»	Новая строка в общем списке
Повторное чтение	Открытие списка на втором ПК	Та же строка появляется у второго клиента
Отметка выполнения	Выбрана добавленная задача	Перед названием показано [x]
Изменение названия	Выбрана задача «Подготовить курсовую»	Новое название сохраняется в базе
Поиск	Введено слово «курсовую»	Отображаются записи с совпадением
Фильтр	Выбрано «Выполнено»	Показаны только выполненные задачи
Статистика	Три задачи, одна выполнена	Доля выполнения составляет 33%
Экспорт в TXT	Две задачи с разными отметками	В выбранном файле записаны обе задачи
Возврат в работу	Выбрана выполненная задача	Перед названием показано [ ]
Неверный ввод	Пустая строка или пробелы	Показана подсказка; запись не создаётся
Удаление	Выбрана существующая задача	Строка исчезает из общего списка
Отсутствие связи	Компьютер-сервер выключен	В строке состояния указано, что соединение отсутствует

Приложение Б

Порядок проверки на двух компьютерах

Для демонстрации на одном компьютере запускают `START.bat`: открывается сервер и два независимых окна клиента. Для проверки двух физических ПК на первом запускают `SERVER_ONLY.bat`, а на втором `CLIENT_OTHER_PC.bat` и вводят локальный IPv4-адрес первого компьютера.

При проверке на двух физических ПК оба компьютера должны находиться в одной локальной сети. Для определения IPv4-адреса на первом ПК выполняют команду `ipconfig`. Если второй компьютер не подключается, проверяют адрес сервера и правила брандмауэра для порта 8000.

После подключения на первом ПК создают задачу «Подготовить курсовую». На втором ПК проверяют появление записи после обновления списка. Затем исправляют название, меняют отметку выполнения и проверяют изменение на первом ПК. На каждом клиенте проверяют поиск, фильтр и показатели страницы «Статистика». Также нажимают «Сохранить TXT», выбирают файл и открывают его в Блокноте Windows. В завершение удаляют задачу. Для проверки сохранения создают новую запись, перезапускают сервер и убеждаются, что запись осталась в списке. Для проверки ошибки соединения временно останавливают сервер, ждут сообщения об отсутствии связи, затем запускают сервер снова и проверяют восстановление списка.

Дата окончания курсового проекта: _____

Подпись автора: _____